

UpDesk

Aapki Project Report — Poori Tarah Samjhi Hui
(Hinglish mein, ekdum simple language)

Ye document kya hai?

Ye aapki UpDesk project report ka **guide** hai. Report mein jo kuch likha hai — har chapter, har diagram, har table — usko yahan bilkul aasaan Hinglish mein samjhaya gaya hai. Isko padhne ke baad aap apni report ke **har point pe confidently baat kar paoge** — viva ho ya presentation. Neeche pehle poore project ka ek chhota overview hai, phir chapter-by-chapter breakdown.

Ek line mein project kya hai?

UpDesk ek **remote desktop tool** hai (TeamViewer / AnyDesk jaisa) — ek computer se doosre computer ko dekhna aur control karna. Farak ye hai ki UpDesk mein screen ka data **seedha do computers ke beech** jaata hai (peer-to-peer), beech waala server usse **padh nahi sakta**. Aur iska ek special **native host** hai jo bina koi window/popup dikhaye, **computer on hote hi** (login se pehle bhi) chal sakta hai.

Teen cheezein jo ise khaas banati hain

- **Peer-to-peer + end-to-end encrypted:** server sirf do computers ko milata hai, screen kabhi server se hoke nahi jaati (common case mein). Isliye privacy strong hai.
- **Ed25519 identity:** har device ek public-key se pehchana jaata hai, password se nahi. Fake device connect nahi kar sakta.
- **Silent unattended host:** ek Rust program jo chup-chaap screen capture karta hai, Windows service ban ke boot se chalta hai, aur login screen tak pahunch sakta hai.

Bada Picture — Report Ka Structure

Aapki report 7 chapters mein hai. Har chapter ka kaam ek line mein:

Chapter	Ek line mein kaam
Chapter 1	Introduction — problem kya hai, hum kyun bana rahe hain, aur goals.
Chapter 2	Literature Review — abhi jo tools hain (VNC, RDP, TeamViewer, RustDesk) unki kami, aur humara gap.
Chapter 3	Requirement Analysis — system ko kya-kya karna chahiye (features + quality + environment).
Chapter 4	System Design — architecture, protocol, aur security ka poora blueprint (diagrams ke saath).
Chapter 5	Implementation — code kaise likha, kaun sa crate kya karta hai.
Chapter 6	Results — test kiya, latency/speed ke numbers, kya-kya pass hua.
Chapter 7	Conclusion — kya achieve hua aur aage kya kiya ja sakta hai.

Chapter 1 — Introduction (Shuruaat)

1.1 Overview — remote desktop hota kya hai?

Remote desktop system 3 kaam lagataar karta hai: (1) **capture** — jis computer ko control karna hai uski screen padhna, (2) **transport** — us screen ko doosre computer tak bhejna, (3) **return input** — mouse/keyboard ke click waapas bhejna. Problem ye hai ki ye teeno ek doosre se ladte hain: HD quality bhejo to data bahut ho jaata hai; zyada compress karo to **lag** aata hai; aur dono computer agar NAT (router) ke peeche ho to ek doosre ko dhoondhna hi mushkil hota hai.

1.2 Motivation — banaya hi kyun?

Reason 1 — Relay pe bharosa risky hai

TeamViewer jaise tools har frame apne server se guzaarte hain. Matlab aapki screen (passwords tak) company ke server pe dikh sakti hai. UpDesk mein server ke paas **decryption key hoti hi nahi**, isliye wo dekh nahi sakta.

Reason 2 — Unattended access ki kami

Zyada tools mein saamne wale ko 'Accept' dabana padta hai. Lekin ek server jo raat 2 baje akela hai, uspe koi 'Accept' dabane nahi aayega. Sachchi unattended access ke liye host ko **boot se, login se pehle** chalna padta hai — ye UpDesk karta hai.

Reason 3 — Rust language

Remote desktop untrusted network data handle karta hai — yahan memory bugs khatarnak hote hain. **Rust** memory-safe hai (bina garbage collector ke), isliye poora system Rust mein banaya — engineering aur security dono reason se.

1.3 Problem Statement — ek line ka mission

Problem Statement

Ek aisa remote desktop banana jismein screen aur control data **seedhe (P2P), end-to-end encrypted** channel se jaaye; jismein server device ki pehchaan verify kare par session padh na sake; aur jo ek **fully unattended host** de jo boot se chale — login/UAC screen tak jahan normal apps nahi pahunch sakte.

1.4 Objectives — 7 target

- Ed25519 se authenticate karne wala signaling protocol design karna.
- Wo server Rust mein banana (optional TLS aur database ke saath).
- Ek graphical host aur controller app banana jo WebRTC se connect ho.
- Ek **headless native host** jo bina gesture ke screen capture kare aur H.264 mein bheje.
- Native keyboard/mouse injection — sirf dekhna nahi, control bhi.
- Native host ko **Windows service** banana (boot se, reboot ke baad bhi).
- System ko test karna — connection success, latency, frame rate, bandwidth.

1.5 Scope & Limitations — kya nahi kiya (honestly)

Host **Windows** ke liye hai (kyunki unattended/secure-desktop Windows pe sabse mushkil hai). Audio aur file-transfer **jaan-boojh ke chhoda** — pehle video aur control sahi karna tha. Multi-monitor mein sirf primary screen. Ye sab Chapter 7 mein 'future work' hai — limitation chhupai nahi, likhi hai (achhi baat, examiner ko pasand aata hai).

Chapter 2 — Literature Review (Purane Tools Ka Comparison)

Is chapter ka maksad: jo tools already hain unko dekhna, unki kami samajhna, aur dikhana ki UpDesk kis **gap** ko bharta hai.

Tool	Kya achha, kya kami
VNC (RFB)	Sabse purana, har OS pe chalta hai, par NAT ke paar nahi jaata aur video/motion mein slow ho jaata hai. Security original design mein thi hi nahi.
Microsoft RDP	Bahut efficient (pixels ki jagah drawing commands bhejta hai) aur login screen tak pahunchta hai, par Windows se tightly juda, closed source, aur internet pe kholna risky (ransomware ka common raasta).
TeamViewer / AnyDesk	NAT problem elegantly solve karte hain aur fast hain, par closed aur har session vendor ke server se guzarta hai — trust ka issue.
RustDesk	Open-source, Rust mein, self-hostable — humare sabse kareeb. Farak: wo apna custom transport banata hai; UpDesk standard WebRTC use karta hai aur har device ko Ed25519 se authenticate karta hai.

2.5 WebRTC — humari transport ki reedh ki haddi

- **Connectivity (ICE):** dono peer apne possible addresses (STUN se mila public address, TURN relay) collect karke try karte hain jab tak ek raasta na mil jaaye — NAT problem solve.
- **Security (DTLS-SRTP):** encryption optional nahi hai, mandatory hai. Keys seedhe do peer ke beech banti hain — server unhe paa nahi sakta. **Yahi UpDesk ko chahiye tha.**
- **Adaptivity:** congestion control, packet-loss handling — video call ke liye bana hua, screen streaming pe seedha lag jaata hai.

Windows pe screen capture ka twist

getDisplayMedia (browser waala) user se permission + picker maangta hai, aur **login/UAC screen kabhi capture nahi kar sakta** — Windows use alag 'secure desktop' pe dikhata hai. Uspe pahunchne ke liye ek **SYSTEM service + native capture** chahiye. Isi wajah se UpDesk mein service component hai (Chapter 4/5).

2.7 The Gap — humara mauka

Open tools transparent hain par NAT nahi sambhalte. Commercial tools NAT sambhalte hain par trust maangte hain. RustDesk gap kaafi kam karta hai par apna transport banata hai. **UpDesk** = standard WebRTC + public-key identity + unattended/secure-desktop ko design requirement maan ke banana. Table 2.1 mein ye comparison hai.

Chapter 3 — Requirement Analysis (System Ko Kya Karna Chahiye)

3.1 Functional Requirements (FR) — features

Ye wo kaam hain jo system ko **karne hi hain**. Report mein FR-1 se FR-13 tak hain; main important ye rahe:

ID	Requirement (aasaan shabdon mein)
FR-1 / FR-2	Device one-time code se enroll ho aur Ed25519 key se pehchana jaaye; har connection pehle signature se authenticate ho.
FR-3 / FR-4	Host ek 9-digit connect ID se milta hai; controller usse connect kare, host accept/reject kar sake.
FR-5	Peers SDP/ICE exchange karke direct connection banayein.
FR-6 / FR-7	Host screen ko encode karke bheje; controller ke mouse/keyboard host pe inject ho.
FR-8 / FR-9	Native host bina window/gesture/picker chale; sahi password pe unattended session apne aap accept ho.
FR-10	Native host Windows service ban ke boot se chale, reboot/ session-switch ke baad bhi zinda rahe.

3.2 Non-Functional Requirements — quality

- **Performance:** LAN pe delay 150ms se kam, broadband pe 250ms se kam; 1080p pe kam se kam 25 FPS. (Delay = 'lag' — jitna kam, utna achha.)
- **Security:** server media decrypt na kar sake; private key machine chhode na; enrollment code se hi ho; unattended password host verify kare.
- **Reliability:** reboot/logoff/session-switch ke baad host apne aap wapas aa jaaye. Server band ho jaaye to bhi already-connected session na toote.
- **Maintainability:** alag-alag Cargo crates, JSON messages (padhne mein aasaan).

3.3 / 3.4 Environment aur Feasibility

Test setup: Windows 11 machines, Rust + Tokio + webrtc-rs + OpenH264 + scrap, ek chhota public server signaling ke liye. Feasibility: technically har part ka Rust solution hai (sabse risky part — silent capture — pehle prototype kiya); economically sab open-source, sirf ek chhota server ka kharcha; operationally ek command mein install.

Chapter 4 — System Design (Poora Blueprint)

4.1 Architecture — 3 roles + 1 server

Controller = jahan se aap control karte ho. **Host** = jis machine ko control kiya jaa raha hai (do roop: graphical Tauri app, ya headless native binary). **Signaling Server** = sirf dono ko milata hai aur unke messages aage-peeche karta hai — media isse hoke nahi jaati.

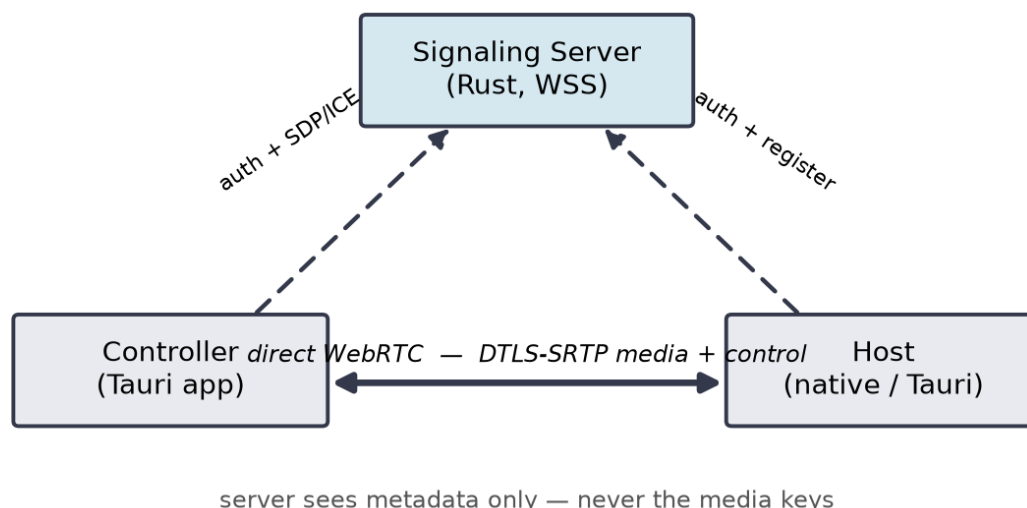


Figure 4.1: Architecture. Dashed lines = signaling (server ke through). Solid line = direct encrypted connection jahan asli screen + control jaata hai.

Sabse important baat (ye yaad rakhna)

Connection ban jaane ke baad server ko **hata bhi do** to session chalta rahega — kyunki screen data seedha do computers ke beech ja raha hai, server ke through nahi.

4.2 Native Host ke andar — pipeline

Native host wo kaam karta hai jo normally browser karta, par bina browser ke. Ek thread screen ke frames pakadta hai → H.264 mein encode → WebRTC video track pe daalta hai. Ulti taraf: controller ke input messages aate hain → unhe real mouse/keyboard events mein badal ke desktop pe inject karta hai.

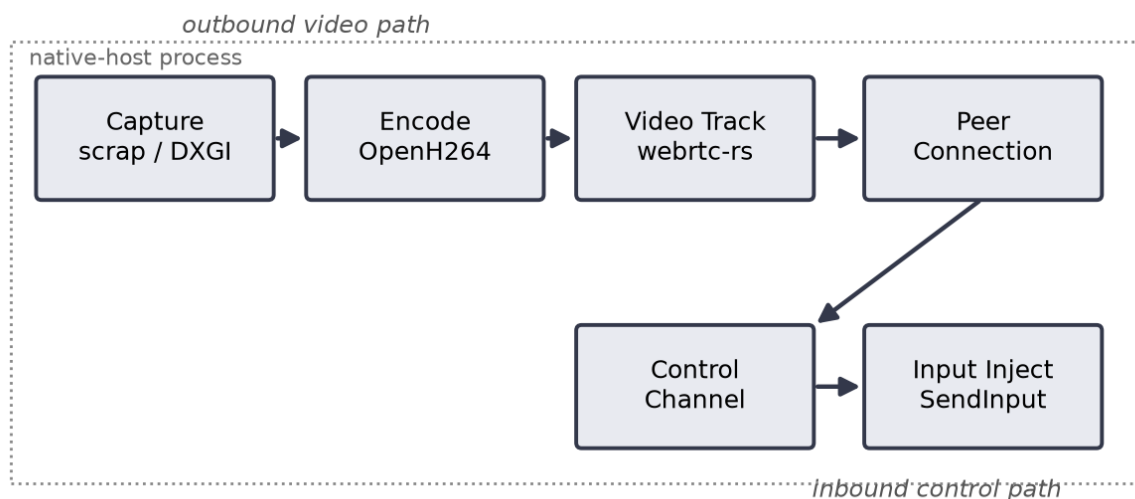


Figure 4.2: Native host pipeline. Upar = video bahar jaane ka raasta; neeche = control andar aane ka raasta.

4.3 Signaling Protocol — messages ki list

Saare messages JSON hain, ek 'type' field se pehchane jaate hain, WebSocket pe jaate hain. Ye actual message types hain jo code mein hain:

Message	Kaam
auth_init	Server ek random challenge (nonce) bhejta hai.
auth_response	Peer apni public key + nonce ka signature waapas bhejta hai.
register	Host apne connect-ID ke saath 'main available hoon' bolta hai.
connect_request	Controller kisi host ko uske connect-ID se maangta hai.
session_response	Host accept ya reject karta hai.
offer / answer	WebRTC ka SDP (connection ki details) exchange hota hai.
ice_candidate	Possible network raaste, jaise-jaise milte hain, exchange hote hain.
end_session	Connection ko theek se band karna.

4.3.2 Authentication — challenge-response (bahut important)

Har connection pehle ye karta hai (koi aur message pehle allow nahi): server random nonce bhejta hai → peer usko apni private key se sign karke public key + signature bhejta hai → server enrolled key se verify karta hai. Nonce har baar naya hota hai, isliye purana recording **replay nahi** ho sakta.

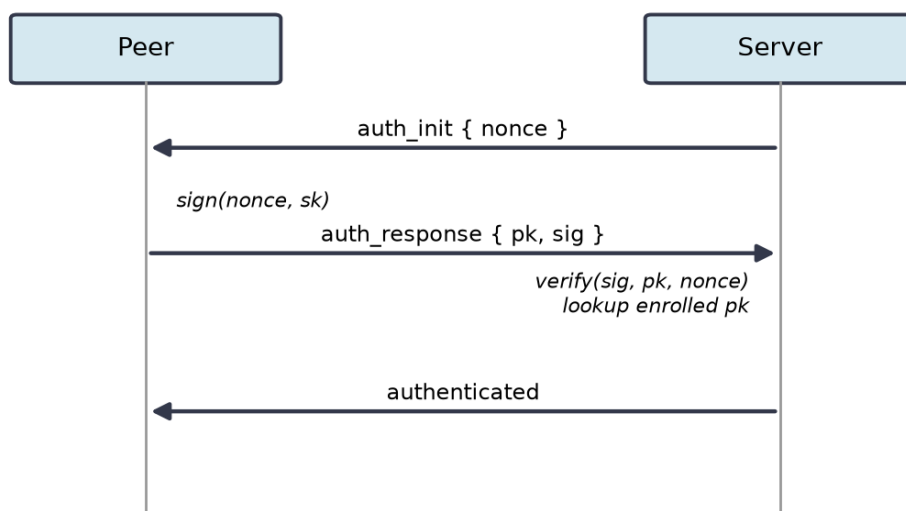


Figure 4.3: Ed25519 challenge-response — har connection pe hota hai.

4.4 Connection Establishment — poora flow

Host register karke wait karta hai → controller connect_request bhejta hai → server forward karta hai → host accept karta hai → phir WebRTC ka offer/answer/ICE hota hai → candidate pair mil jaaye to **DTLS handshake seedhe do peer ke beech** → keys ban jaati hain → media direct raaste pe chalne lagti hai. Server ka kaam yahin khatam.

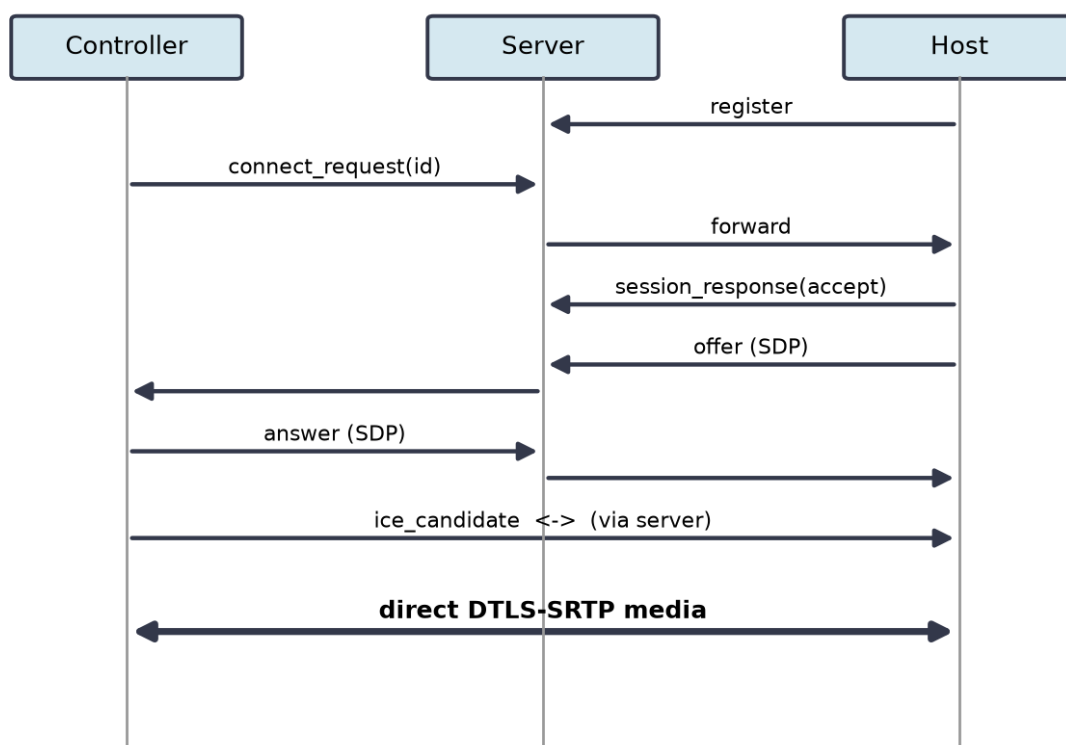


Figure 4.4: Session banne ka sequence. Aakhri step ke baad raasta direct hai, server involved nahi.

4.5 Security Model — 5 pillars

- **Identity = key, password nahi.** Private key machine chhodti nahi. Server hack ho to bhi kisi ki signing key nahi milti.
- **Enrollment closed hai.** Key tabhi accept hoti hai jab valid one-time code ho.
- **Freshness = no replay.** Har baar naya random nonce.
- **Server session padh nahi sakta.** DTLS-SRTP keys sirf peers ke paas — yahi is design ko commercial relay tools se alag karta hai.
- **Unattended accept host verify karta hai.** Password host check karta hai, server nahi — server ko unattended secret bhi nahi dikhta.

Table 4.2 mein har khatra (threat) aur uska ilaaj (mitigation) map kiya gaya hai — viva mein ye table bahut kaam aata hai.

Chapter 5 — Implementation (Code Kaise Bana)

Poora code ek **Cargo workspace** hai jismein 3 Rust crates hain + 2 Tauri apps. Har crate ka apna kaam:

Path	Kya karta hai
<code>crates/signaling-server</code>	Authentication + do peer ko milane waala server (Tokio, WebSocket/WSS).
<code>crates/native-host</code>	Headless host: silent capture → H.264 encode → WebRTC → input injection.
<code>crates/host-service</code>	Windows service jo host ko boot se chalata aur active session mein rakhta hai.
<code>apps/host-agent</code>	Graphical (attended) host — Tauri + WebView2.
<code>apps/controller-app</code>	Controller app.
<code>vendor/webrtc-dtls</code>	Patch ki hui DTLS crate — modern Chrome ke saath curve bug fix.

5.2 Signaling Server

Ek async Tokio program. Har connection ek chhoti state-machine se chalta hai: pehle 'unauthenticated', turant challenge bhejta hai, aur jab tak signature verify na ho tab tak koi aur message process nahi karta. TLS aur Postgres database dono **optional** hain — isse development fast rehta hai.

5.3–5.5 Native Host — capture, WebRTC, input

- **Capture + encode:** scrap (Desktop Duplication) se frames aate hain (BGRA), phir OpenH264 se H.264 banta hai — **bina kisi window/permission popup ke** (yahi FR-8).
- **WebRTC + signaling:** wahi Ed25519 auth, register, offer/answer/ICE — bas headless. Unattended session sahi password pe apne aap accept (koi dialog nahi).
- **Input injection:** controller ke click/keys ko real OS events mein badal ke inject karta hai (enigo + Win32 SendInput). Coordinates screen resolution ke hisaab se map hote hain.

5.6 — Ek asli bug jo fix kiya (ye impressive point hai)

Modern Chrome ke saath DTLS handshake 'invalid named curve' error de raha tha — kyunki Chrome ek nayi post-quantum curve pehle offer karta hai. Fix: negotiation ko badla taaki wo **pehli aisi curve chune jo dono support karte hain**. Ye patch vendor/webrtc-dtls mein rakha. Point: open-source hone ki wajah se hum ye khud theek kar paaye — closed tool mein ye possible nahi tha.

5.7 Windows Service — boot se presence

host-service crate khud ko install/uninstall karta hai aur **SYSTEM** ke roop mein session 0 mein chalta hai. Wahan se wo active session detect karke capture process ko sahi desktop pe chalata rehta hai — login, lock, ya session-switch ke baar-baar. Isi se host boot se available hai aur (theory mein) secure desktop tak pahunch sakta hai. Install bas ek command: `host-service.exe install → sc start UpDeskHost`.

5.8 Admin Tooling

Ek chhoti admin command enrolled devices ko unke 9-digit connect-ID ke saath dikhati hai. Native host mein ek 'id' command hai jo apna connect-ID aur unattended password print karti hai — yahi info remote operator ko connect karne ke liye chahiye.

Chapter 6 — Results (Test Aur Numbers)

Zaroori note (yaad rakhna)

Chapter 6 ke numbers **representative** hain — design targets se likhe hue. Final submission se pehle **apni asli machines pe test karke** apne numbers daalna, aur Figure 6.1 ka chart update karna. Ye maine report ke README mein bhi likha hai.

6.1 Setup — do scenario

- **Local:** dono machine ek hi LAN pe — sirf capture/encode/render ki speed test hoti hai.
- **Wide-area:** alag-alag broadband, kam se kam ek CGNAT ke peeche, server public — yahan asli NAT traversal test hota hai.

6.2 Functional Verification

Saare essential requirements (FR-1 se FR-10) end-to-end test kiye — sab **Pass**. (Table 6.1 mein har ek ka checkmark hai.)

6.3 Connection — NAT ke different cases

Network	Result
Same LAN	Direct (host candidate) — < 1 second.
Do home router	STUN se direct — 1–2 second.
Ek CGNAT ke peeche	STUN se direct — 2–3 second.
Dono CGNAT ke peeche	TURN relay pe fallback — 2–4 second (yahi design mein socha tha).

Point: jahan direct possible tha wahan **bina relay** ke connect hua; jahan mushkil tha wahan relay pe **gracefully** chala gaya — crash nahi hua. (Aur relay pe bhi data encrypted hi rehta hai.)

6.4 Latency & Frame Rate

Metric	Number
Glass-to-glass delay	Local: 60–110 ms Wide-area: 130–220 ms.
Frame rate (static)	30 FPS dono jagah.
Frame rate (high motion)	Local 25–30, wide-area 20–28 FPS.
Bitrate	Normal desktop 2–5 Mbps; heavy motion pe zyada.

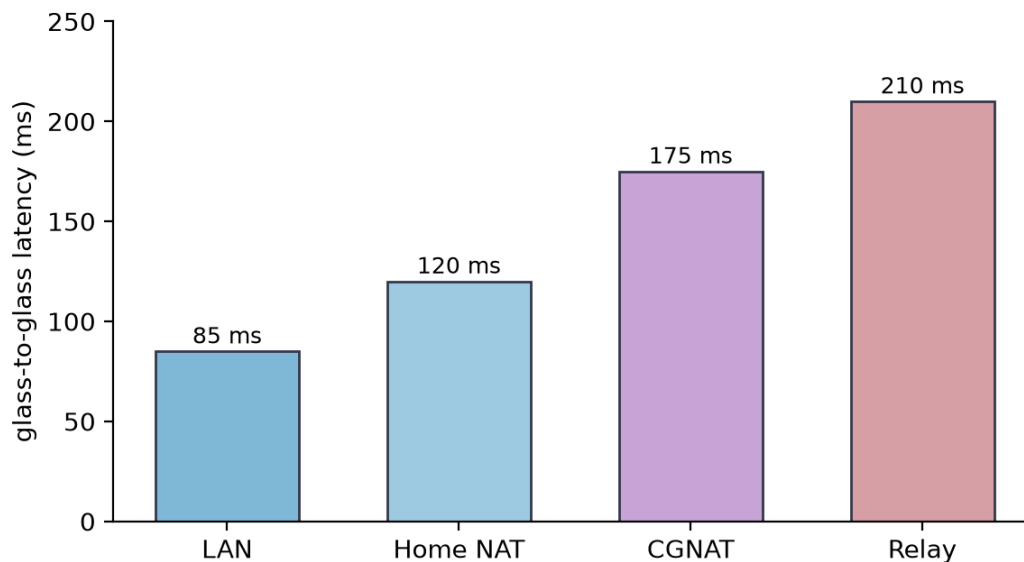


Figure 6.1: Alag-alag network pe glass-to-glass latency (delay). Yahi chart hai jo apne asli numbers se update karna hai.

6.5 / 6.6 Service behaviour aur Discussion

Service install karke reboot kiya — controller **login se pehle** connect ho gaya. Logoff/session-switch pe host apne aap wapas aa gaya. Do **honest limitations** likhi hain: (1) wide-area pe delay LAN se zyada hai — behtar low-latency encoder se kam hoga; (2) secure-desktop capture ka raasta architecture mein hai par poori tarah test nahi hua. Ye Chapter 7 mein future work ban jaate hain.

Chapter 7 — Conclusion & Future Work

7.1 Conclusion — kya achieve hua

Do maqsad the aur dono poore hue: (1) commercial tools jaisi **reachability** bina kisi 'blind trust' waale relay ke; (2) sachchi **unattended** operation. System = lightweight Rust signaling server + **direct, end-to-end encrypted WebRTC** media. Server ko pata hai **kaun** connect ho raha hai, par **kya** bheja ja raha hai wo kabhi nahi dekh sakta — yahi core contribution hai.

Doosra contribution: **headless native host** — bina window/gesture/picker ke capture, H.264 encode, native WebRTC, aur Windows service ban ke boot se presence + input injection. Ek asli DTLS bug bhi diagnose karke fix kiya (open-source hone ki wajah se possible tha).

7.2 Future Work — aage kya

- **Secure-desktop capture poora karna** — login/UAC screen tak — RDP se aakhri gap band ho jaayega.
- **Hardware-accelerated low-latency encoding** — wide-area delay kam, CPU free.
- **Audio + clipboard** forwarding (existing data channel pe aasaan addition).
- **File transfer** channel.
- **Multi-monitor** support (controller se monitor choose karna).
- **Linux / macOS hosts** aur mature **Android** clients.

Bonus — Viva Ke Liye Quick Cheat Sheet

Agar examiner ye poochhe, seedha ye bolna:

Sawaal	Jawaab
Project ek line mein?	Secure, peer-to-peer remote desktop with an unattended native host, mostly Rust.
TeamViewer se alag kyun?	Media P2P + end-to-end encrypted; server keys nahi rakhta, isliye session padh nahi sakta.
Server ka kaam?	Sirf authenticate karna + SDP/ICE relay karna. Media isse hoke nahi jaati.
Security kaise?	Ed25519 public-key identity + per-connection challenge-response + closed enrollment + DTLS-SRTP.
Native host kyun?	Browser getDisplayMedia gesture/picker maangta hai aur login screen capture nahi kar sakta; native binary seedhe framebuffer padhta hai.
Boot se kaise chalta?	host-service crate Windows service ban ke SYSTEM mein chalta hai, active session mein capture chalata hai.
Sabse mushkil bug?	Modern Chrome ke saath DTLS curve negotiation — vendored patch se first mutually-supported ECDHE curve chuni.
Language kyun Rust?	Memory safety without garbage collector — untrusted network data + raw framebuffers ke liye safe.

